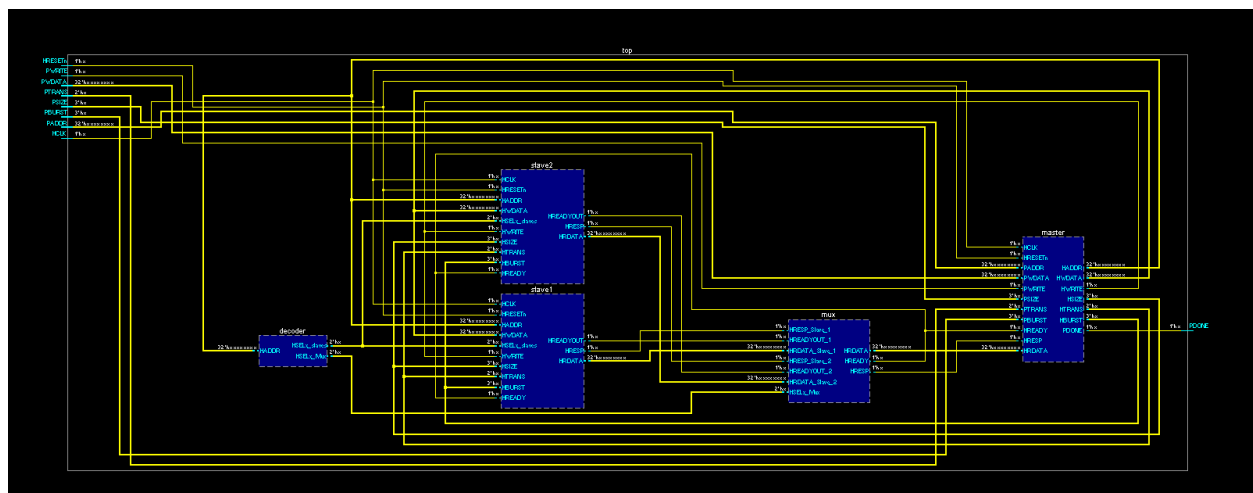
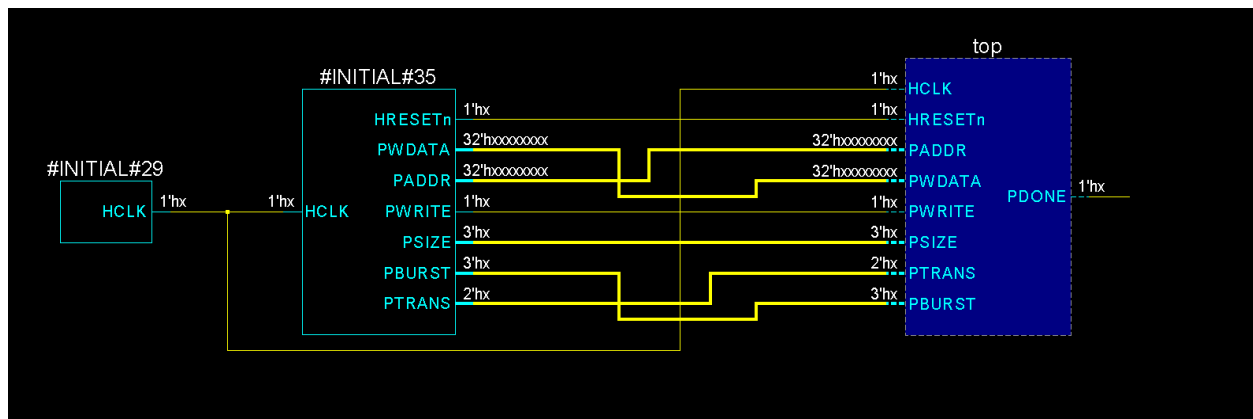
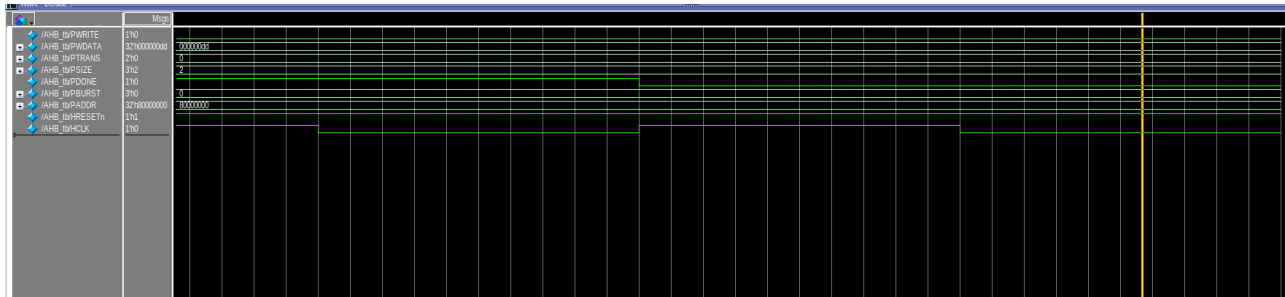


Submitted by: Kashish Shaikh

Task 1:



```

# Compile of AHB_Lite_Master.v was successful.
# Compile of AHB_tb.v was successful.
# Compile of AHB_TOP.v was successful.
# Compile of AHB_Slave_1.v was successful.
# Compile of AHB_Slave_2.v was successful.
# 7 compiles, 0 failed with no errors.
vsim work.AHB_tb
# vsim work.AHB_tb
# Start time: 15:56:44 on Aug 18, 2026
# ** Note: (vsim-3812) Design is being optimized...
# Loading work.AHB_tb(fast)
QuestaSim> vsim -voptargs="+acc" work.AHB_tb
# End time: 15:57:19 on Aug 18, 2026, Elapsed time: 0:00:35
# Errors: 0, Warnings: 3
# vsim -voptargs="+acc" work.AHB_tb
# Start time: 15:57:19 on Aug 18, 2026
# ** Note: (vsim-3812) Design is being optimized...
# ** Note: (vopt-143) Recognized 1 FSM in module "AHB_Master(fast)".
# ** Note: (vopt-143) Recognized 1 FSM in module "AHB_Slave_2(fast)".
# Loading work.AHB_tb(fast)
# Loading work.AHB_TOP(fast)
# Loading work.AHB_Master(fast)
# Loading work.AHB_Decoder(fast)
# Loading work.AHB_Slave_1(fast)
# Loading work.AHB_Slave_2(fast)
# Loading work.AHB_MUX(fast)
add wave -position insertpoint \
sim:/AHB_tb/PWRITE \
sim:/AHB_tb/PWDATA \
sim:/AHB_tb/PTRANS \
sim:/AHB_tb/PSIZE \
sim:/AHB_tb/PODR \
sim:/AHB_tb/PBURST \
sim:/AHB_tb/PADDR \
sim:/AHB_tb/HRESETn \
sim:/AHB_tb/HCLK
VSIW> run -all
#
# [50000 ns] --- TEST CASE 1: Single Write Transfer ---
#
# [110000 ns] --- TEST CASE 2: Single Read Transfer ---
#
# [170000 ns] --- TEST CASE 3: Write with Wait-State ---
#
# [230000 ns] --- TEST CASE 4: Burst Transfer (INCR) ---
#
# [350000 ns] --- TEST CASE 5: Invalid Address Transfer ---
#
# [430000 ns] --- All 5 Test Cases Executed Successfully ---
# ** Note: $stop : /home/cl4/Downloads/ahb/aKitBh/AHB_tb.v(150)
# Time: 430 ns Iteration: 0 Instance: /AHB_tb

```

AHB_MASTER:

```

module AHB_Lite_Master (
    // Global Signals
    input      HCLK,
    input      HRESETn,

    // Processor Signals (Testbench Interface)
    input      [31:0] PADDR,
    input      [31:0] PWDATA,
    input      PWRITE,
    input      [2:0] PSIZE,
    input      [1:0] PTRANS,
    input      [2:0] PBURST,

    // Transfer Response from Slave / MUX
    input      HREADY,      // Wait-state indicator (0 = Wait, 1 = Ready)
    input      HRESP,
    input      [31:0] HRDATA,

    // AHB Bus Outputs
    output reg [31:0] HADDR,
    output reg [31:0] HWDATA,
    output reg      HWRITE,
    output reg [2:0] HSIZE,
    output reg [1:0] HTRANS,
    output reg [2:0] HBURST,
    output reg      PDONE

);

// AHB State Definitions
localparam IDLE   = 2'b00;
localparam BUSY   = 2'b01;
localparam NONSEQ = 2'b10;
localparam SEQ     = 2'b11;

reg [1:0] cs, ns;
reg [31:0] HWDATA_reg;

// 1. State Memory (Respects HREADY for Wait States)
always @(posedge HCLK or negedge HRESETn) begin
    if (!HRESETn)
        cs <= IDLE;

```

```

        if (!HRESETn)
            cs <= IDLE;
        else if (HREADY) // Wait-state hone par state freeze rahegi
            cs <= ns;
    end

    // 2. Next State Logic
    always @(*) begin
        case (cs)
            IDLE: begin
                if (PTRANS == 2'b10)
                    ns = NONSEQ;
                else
                    ns = IDLE;
            end

            BUSY: begin
                if (PTRANS == 2'b11)
                    ns = SEQ;
                else if (PTRANS == 2'b10)
                    ns = NONSEQ;
                else if (PTRANS == 2'b00)
                    ns = IDLE;
                else
                    ns = BUSY;
            end

            NONSEQ: begin
                if (PTRANS == 2'b11)
                    ns = SEQ;
                else if (PTRANS == 2'b00)
                    ns = IDLE;
                else if (PTRANS == 2'b10 && PBURST == 3'b000)
                    ns = NONSEQ;
                else
                    ns = SEQ;
            end

            SEQ: begin
                if (PTRANS == 2'b00)
                    ns = IDLE;

```

```

                else if (PTRANS == 2'b10)
                    ns = NONSEQ;
                else
                    ns = SEQ;
            end

            default: ns = IDLE;
        endcase
    end

    // 3. Address & Control Output Logic (Gated by HREADY)
    always @(posedge HCLK or negedge HRESETn) begin
        if (!HRESETn) begin
            HADDR      <= 32'b0;
            HWDATA_reg <= 32'b0;
            HWRITE      <= 1'b0;
            HSIZE       <= 3'b000;
            HTRANS      <= 2'b00;
            HBURST      <= 3'b000;
        end
        else if (HREADY) begin // Agar Slave ready na ho (HREADY=0), address/control hc
            case (cs)
                IDLE: begin
                    HADDR      <= 32'b0;
                    HWDATA_reg <= 32'b0;
                    HWRITE      <= 1'b0;
                    HSIZE       <= 3'b000;
                    HTRANS      <= 2'b00;
                    HBURST      <= 3'b000;
                end

                BUSY, NONSEQ: begin
                    HADDR      <= PADDR;
                    HWDATA_reg <= PWDATA;
                    HWRITE      <= PWRITE;
                    HSIZE       <= PSIZE;
                    HTRANS      <= PTRANS;
                    HBURST      <= PBURST;
                end

                SEQ: begin

```

```

HWRITE      <= PWRITE;
HSIZE       <= PSIZE;
HTRANS      <= PTRANS;
HBURST      <= PBURST;

if (PBURST == 3'b001) begin // Incremental Burst
    case (PSIZE)
        3'b000: begin // 8-bit
            HADDR      <= HADDR + 1'b1;
            HWDATA_reg <= {24'h000000, PWDATA[7:0]};
        end
        3'b001: begin // 16-bit
            HADDR      <= HADDR + 2'd2;
            HWDATA_reg <= {16'h0000, PWDATA[15:0]};
        end
        3'b010: begin // 32-bit
            HADDR      <= HADDR + 3'd4;
            HWDATA_reg <= PWDATA;
        end
        default: begin
            HADDR      <= HADDR + 3'd4;
            HWDATA_reg <= PWDATA;
        end
    endcase
end
else begin // Single / Non-burst
    HADDR      <= PADDR;
    HWDATA_reg <= PWDATA;
end
end
endcase
end

// 4. Data Phase (HWDATA generation aligned with HREADY)
always @(posedge HCLK or negedge HRESETn) begin
    if (!HRESETn)
        HWDATA <= 32'b0;
    else if (HREADY)
        HWDATA <= HWDATA_reg;
end

// 5. Transfer Complete Indicator
always @(*) begin
    if ((cs == NONSEQ || cs == SEQ) && ns == IDLE && HREADY)
        PDONE = 1'b1;
    else
        PDONE = 1'b0;
end

endmodule

```

Task 02:

```
`timescale 1ns/1ps

module AHB_tb ();
    // Processor / Interface Testbench Signals
    reg        HCLK;
    reg        HRESETn;
    reg [31:0] PADDR;
    reg [31:0] PWDATA;
    reg        PWRITE;
    reg [2:0]  PSIZE;
    reg [1:0]  PTRANS;
    reg [2:0]  PBURST;
    wire       PDONE;

    // Instantiate AHB_TOP Module
    AHB_TOP top (
        .HCLK(HCLK),
        .HRESETn(HRESETn),
        .PADDR(PADDR),
        .PWDATA(PWDATA),
        .PWRITE(PWRITE),
        .PSIZE(PSIZE),
        .PTRANS(PTRANS),
        .PBURST(PBURST),
        .PDONE(PDONE)
    );

    // Clock Generation: 50 MHz (20ns Period)
    initial begin
        HCLK = 0;
        forever #10 HCLK = ~HCLK;
    end

    // Simulation Sequence (Task 2: 5 Comprehensive Test Cases)
    initial begin
        // -----
        // System Reset Initialization
        // -----
        HRESETn = 0;
        PADDR   = 32'h0;
        PWDATA   = 32'h0;

        PWRITE   = 0;
        PSIZE    = 3'b000;
        PTRANS   = 2'b00;
        PBURST   = 3'b000;
        #40;
        HRESETn = 1;
        @(posedge HCLK);

        // =====
        // Test Case 1: Single Write Transfer (No Wait-state)
        // Target: Slave 1 (Addr: 0x00000004), Size: 32-bit (0xDEADBEEF)
        // =====
        $display("\n[%0t ns] --- TEST CASE 1: Single Write Transfer ---", $time);
        @(posedge HCLK);
        PADDR <= 32'h00000004; // Address Phase
        PWDATA <= 32'hDEADBEEF; // Data Phase value
        PWRITE <= 1'b1;        // Write
        PSIZE  <= 3'b010;      // 32-bit
        PTRANS <= 2'b10;       // NONSEQ
        PBURST <= 3'b000;      // SINGLE

        @(posedge HCLK);
        PTRANS <= 2'b00;        // Move to IDLE (Data Phase completes)
        @(posedge HCLK);

        // =====
        // Test Case 2: Single Read Transfer (No Wait-state)
        // Target: Slave 1 (Addr: 0x00000004), Read back 32-bit data
        // =====
        $display("\n[%0t ns] --- TEST CASE 2: Single Read Transfer ---", $time);
        @(posedge HCLK);
        PADDR <= 32'h00000004; // Address Phase
        PWRITE <= 1'b0;        // Read
        PSIZE  <= 3'b010;      // 32-bit
        PTRANS <= 2'b10;       // NONSEQ
        PBURST <= 3'b000;      // SINGLE

        @(posedge HCLK);
        PTRANS <= 2'b00;        // IDLE (Read Data sampled)
        @(posedge HCLK);
    end
endmodule
```

```

$display("\n[%0t ns] --- TEST CASE 3: Write with Wait-State ---", $time);
@(posedge HCLK);
PADDR <= 32'h40000000; // Slave 2 base address
PDATA <= 32'hCAFE1234;
PWRITE <= 1'b1;
PSIZE <= 3'b010; // 32-bit
PTRANS <= 2'b10; // NONSEQ
PBURST <= 3'b000;

@(posedge HCLK);
PTRANS <= 2'b00; // IDLE
@(posedge HCLK);

// =====
// Test Case 4: Burst Transfer (INCR4 - 4 transfers, 8-bit)
// Target: Slave 1 (Addr: 0x00000010 to 0x00000013)
// =====
$display("\n[%0t ns] --- TEST CASE 4: Burst Transfer (INCR) ---", $time);
@(posedge HCLK);
// Beat 1 (NONSEQ)
PADDR <= 32'h00000010;
PDATA <= 32'hAA;
PWRITE <= 1'b1;
PSIZE <= 3'b000; // 8-bit
PTRANS <= 2'b10; // NONSEQ
PBURST <= 3'b001; // INCR

@(posedge HCLK);
// Beat 2 (SEQ)
PDATA <= 32'hBB;
PTRANS <= 2'b11; // SEQ

@(posedge HCLK);
// Beat 3 (SEQ)
PDATA <= 32'hCC;
PTRANS <= 2'b11; // SEQ

@(posedge HCLK);
// Beat 4 (SEQ)
PDATA <= 32'hDD;
PTRANS <= 2'b11; // SEQ

@(posedge HCLK);
PTRANS <= 2'b00; // IDLE
@(posedge HCLK);

// =====
// Test Case 5: Invalid Address with Error Response
// Target: Unmapped region (Addr: 0x80000000 / Slave 3 select)
// =====
$display("\n[%0t ns] --- TEST CASE 5: Invalid Address Transfer ---", $time);
@(posedge HCLK);
PADDR <= 32'h80000000; // Selects unmapped Slave 3 space
PWRITE <= 1'b0; // Read
PSIZE <= 3'b010;
PTRANS <= 2'b10; // NONSEQ
PBURST <= 3'b000;

@(posedge HCLK);
PTRANS <= 2'b00; // IDLE
#40;

$display("\n[%0t ns] --- All 5 Test Cases Executed Successfully ---", $time);
$stop;
end

```

endmodule

AHB_top

```
module AHB_TOP (
    input HCLK,
    input HRESETn,
    input [31:0] PADDR,
    input [31:0] PWDATA,
    input PWRITE,
    input [2:0] PSIZE,
    input [1:0] PTRANS,
    input [2:0] PBURST,
    output PDONE
);
    wire [31:0] HADDR;
    wire [31:0] HWDATA;
    wire HWRITE;
    wire [2:0] HSIZE;
    wire [1:0] HTRANS;
    wire [2:0] HBURST;
    wire [1:0] HSELx_slaves;
    wire [1:0] HSELx_Mux;
    wire HREADYOUT_1;
    wire HREADYOUT_2;
    wire HRESP_slave_1;
    wire HRESP_slave_2;
    wire [31:0] HRDATA_1;
    wire [31:0] HRDATA_2;
    wire [31:0] HRDATA;
    wire HREADY;
    wire HRESP;

    AHB_Master master (
        .HCLK(HCLK),
        .HRESETn(HRESETn),
        .PADDR(PADDR),
        .PWDATA(PWDATA),
        .PWRITE(PWRITE),
        .PSIZE(PSIZE),
        .PTRANS(PTRANS),

        .PBURST(PBURST),
        .HADDR(HADDR),
        .HWDATA(HWDATA),
        .HWRITE(HWRITE),
        .HSIZE(HSIZE),
        .HTRANS(HTRANS),
        .HBURST(HBURST),
        .HREADY(HREADY),
        .HRESP(HRESP),
        .HRDATA(HRDATA),
        .PDONE(PDONE)
    );

    AHB_Decoder decoder (
        .HADDR(HADDR),
        .HSELx_slaves(HSELx_slaves),
        .HSELx_Mux(HSELx_Mux)
    );

    AHB_Slave_1 slave1 (
        .HCLK(HCLK),
        .HRESETn(HRESETn),
        .HADDR(HADDR),
        .HWDATA(HWDATA),
        .HSELx_slaves(HSELx_slaves),
        .HWRITE(HWRITE),
        .HSIZE(HSIZE),
        .HTRANS(HTRANS),
        .HBURST(HBURST),
        .HREADY(HREADY),
        .HREADYOUT(HREADYOUT_1),
        .HRESP(HRESP_slave_1),
        .HRDATA(HRDATA_1)
    );
```

```

AHB_Slave_2 slave2 (
    .HCLK(HCLK),
    .HRESETn(HRESETn),
    .HADDR(HADDR),
    .HWDATA(HWDATA),
    .HSELx_slaves(HSELx_slaves),
    .HWRITE(HWRITE),
    .HSIZE(HSIZE),
    .HTRANS(HTRANS),
    .HBURST(HBURST),
    .HREADY(HREADY),
    .HREADYOUT(HREADYOUT_2),
    .HRESP(HRESP_Slave_2),
    .HRDATA(HRDATA_2)
);

AHB_MUX mux (
    .HRESP_Slave_1(HRESP_Slave_1),
    .HREADYOUT_1(HREADYOUT_1),
    .HRDATA_Slave_1(HRDATA_1),
    .HRESP_Slave_2(HRESP_Slave_2),
    .HREADYOUT_2(HREADYOUT_2),
    .HRDATA_Slave_2(HRDATA_2),
    .HSELx_Mux(HSELx_Mux),
    .HRDATA(HRDATA),
    .HREADY(HREADY),
    .HRESP(HRESP)
);

```

```
endmodule
```